

CNFP: Optimizing Cloud-Native Network Function Placement with Diffusion Models on the Cloud Continuum

Álvaro Vázquez-Rodríguez, Manuel Fernández-Veiga, Carlos Giraldo-Rodríguez

Abstract—The placement of Cloud-Native Network Functions (CNFs) across the Cloud-Continuum represents a core challenge in the orchestration of current 5G and future 6G networks. The process entails the implementation of interdependent computing tasks, which are structured as Service Function Chains, over distributed cloud infrastructures. This is achieved while satisfying strict resource, bandwidth, connectivity, and end-to-end latency constraints. It is widely acknowledged that classical approaches, including mixed-integer (non)linear programming, heuristics, and reinforcement learning, face practical limitations in terms of scalability, robust constraint handling, and generalization to unseen network conditions. In this study, a diffusion-based theoretical and algorithmic framework for CNF placement is proposed, based on Denoising Diffusion Probabilistic Models (DDPM). The placement process is reconceptualised as a conditional graph-to-assignment generation task. Each scenario is encoded as a heterogeneous graph, capturing infrastructure and service-chain structure. A Graph Neural Network (GNN) denoiser is trained to iteratively refine noisy *CNF-to-cloud* assignment matrices. In order to bias the generation process towards valid deployments, the model incorporates constraint-aware penalties during training. At inference, a multitude of candidate placements are sampled, and the best suboptimal, feasible solution is selected. This enables a controllable trade-off between solution quality and runtime. Extensive experimentation on diverse topologies, incorporating out-of-distribution evaluations with larger instances and shifted constraint regimes, demonstrates that the proposed approach consistently generates feasible solutions with considerably accelerated inference compared to MINLP solvers when available, while maintaining robust feasibility under constraint-tight scenarios. The findings of this study demonstrate the potential of diffusion-based generative modelling as a scalable tool for constrained network placement and embedding in cloud-continuum orchestration.

Index Terms—DDPM, GNN, CNF, Network embedding, Cloud-Continuum.

I. INTRODUCTION

NETWORK operators are undergoing a significant transformation in the deployment of 5G networks. This transformation is characterised by the adoption of software-centric principles and a shift towards cloud-native design methodologies. The transition is marked by a departure from the utilisation of monolithic appliances and VM-based implementations, towards a microservice-oriented paradigm. This

shift is exemplified by the adoption of Network Function Virtualisation (NFV) and containerised network functions, which are designed to enhance operational efficiency and flexibility. In this context, it is becoming increasingly commonplace for 5G system functions to be implemented as software network functions. These functions can be instantiated and scaled on demand, with coordination facilitated by NFV Management and Orchestration (NFV-MANO) frameworks [1], [2]. Concurrently, the transition towards Multi-access Edge Computing (MEC) and fog/edge paradigms facilitates the migration of computation from central clouds to resource-constrained nodes in closer proximity to end users. This development enables the provision of latency-sensitive services at the network edge [3], [4]. This evolution gives rise to a heterogeneous cloud continuum, in which functions may be distributed across both core and edge sites, with the consequence that these sites are subject to diverse resource, connectivity, and operational cost constraints.

In order to deliver end-to-end services, network functions are frequently composed into Service Function Chains (SFCs). In such cases, traffic must traverse an ordered (or partially ordered) set of functions. The collective provision of these functions results in a higher-level network service [5]. Determining the location of required functions across the cloud continuum, whilst satisfying constraints such as node capacity (e.g. CPU/RAM), link bandwidth, and stringent end-to-end delay budgets, gives rise to challenging combinatorial optimisation problems. These problems are widely recognised as difficult and are commonly addressed through mixed-integer optimisation, heuristics/metaheuristics, or learning-based approximations [5], [6]. Exact mixed-integer (and mixed-integer nonlinear) formulations have been shown to provide optimal solutions for small instances. However, as the number of nodes, functions and constraints increases, these formulations scale poorly. This motivates the use of faster heuristics that are tailored to specific scenarios [6]. In recent times, the application of reinforcement learning to cope with dynamism and large state spaces has been explored. However, this typically requires carefully designed reward functions and still struggles to enforce strict feasibility under hard constraints [7].

Recently, solutions to these problems based on diffusion models have begun to emerge as a competitive alternative. Diffusion probabilistic models are a type of machine learning model that learns a data distribution by reversing a gradual noising process. During the training phase, noise is progressively injected into a data sample, while a neural denoiser is

The work done is supported by the Axencia Galega de Innovación (GAIN) (11/IN606D/2023/2547030) and by grant PID2023-148716OB-C31 funded by MCIU/AEI/10.13039/501100011033 (project DISCOVERY).

Álvaro Vázquez-Rodríguez and Manuel Fernández-Veiga are withatlanTTic - I&C Lab - Universidade de Vigo

Álvaro Vázquez-Rodríguez and Carlos Giraldo-Rodríguez are with Centro Tecnológico de Telecomunicaciones de Galicia (GRADIANT), Vigo, Spain

optimised to predict and remove that noise. At the inference phase, generation starts from noise and iteratively denoises to synthesise new samples [8]. This iterative generation is attractive for structured decision-making because it can represent multi-modal solution spaces and, through parallel sampling, produce diverse candidates from which the best feasible solution can be selected under system constraints. Beyond the original DDPM formulation, score-based perspectives unify diffusion and enable alternative samplers and guidance mechanisms that can trade off sample quality and runtime [9]. Recent research has demonstrated the efficacy of graph-conditioned diffusion in solving NP-hard combinatorial problems. This is achieved by learning to denoise structured solutions on graphs and improving quality via sampling and selection [10]. The concept of diffusion as a general solution generator for network optimisation has been proposed in the context of networking. The model in question learns a conditional solution distribution, with sampling concentrating probability mass around high-quality decisions [11], [12]. It is important to note that graph diffusion models are capable of being trained from suboptimal or heuristic datasets, with the result that near-optimal solutions are recovered. This, in turn, has the effect of reducing the reliance on expensive exact solvers [13], [14]. The most closely related work to the problem under discussion is that of Network Diffuser, which applies conditional diffusion to the placement and scheduling of service function chains using inverse demonstration. This highlights the suitability of diffusion for SFC decision generation under constraints [15]. The motivation behind the present study was provided by the results of earlier research. The latter constituted an examination of the cloud continuum, with the objective of determining the feasibility of casting static CNF/SFC placement in this setting as a conditional generation problem. This was achieved by means of a graph-based denoiser, which learned to map infrastructure and service-chain structure to a distribution over placement assignments. The result of this learning process was the enabling of fast inference through sampling and post-selection.

Based on these ideas, the present article proposes a solver based on diffusion models for SFC/CNF placement across the cloud continuum. Specifically, the contributions can be categorised into three distinct aspects. Firstly, a static placement problem for multiple SFCs over a cloud-continuum infrastructure graph is formalised, capturing heterogeneous node capacities and costs, CNF resource demands and processing delays, placement restrictions, and end-to-end feasibility requirements under CPU/RAM, adjacency, bandwidth, and SFC delay constraints. Secondly, a conditional diffusion-based generative solver is proposed, with each instance represented as a heterogeneous graph (cloud nodes, CNF nodes, and SFC edges) and capable of learning to denoise a CNF-to-cloud assignment matrix. A graph neural network (GNN) encoder processes the infrastructure and service-chain structure, while a placement mask enforces forbidden CNF-cloud pairs and additional constraint-aware loss terms bias training towards feasible, high-quality solutions. At inference, multiple samples are generated and the best feasible placement is selected. Thirdly, an extensive experimental investigation was conducted

on 44 scenarios. A GEKKO-based MINLP solver acted as the optimal reference when available, and this was compared against two greedy heuristics and a range of learning-based baselines. To assess scalability and generalisation in cost- and constraint-shifted regimes, out-of-distribution scenarios were employed.

The remainder of the paper is organized as follows. First, in Section II, we present the main prior works related to our contributions. How we model the network of our system, how CNFs and SFCs are defined and the formulation of the problem are presented in Section III. The design and implementation of the solver based on diffusion models and Graph Neural Networks (GNNs) is explained in Section IV, and the explanation of how we evaluate, compare and obtain results to measure our model can be found in Section V. Concluding remarks, actual limitations and future work are given in Section VI.

II. RELATED WORK

The intersection of diffusion models approaches and network optimisation has expanded rapidly in the past years, fuelled by the success of Denoising Diffusion Probabilistic Models (DDPMs) [8] in combinatorial tasks. Surveys on virtual network embedding and service function chain deployment already note a shift from classical heuristics toward learning-based solvers [16], and several works now explore diffusion techniques specifically for resource-allocation and routing problems [12].

One of the earliest framework in this path is [11]. DiffSG trains a continuous-time DDPM conditioned on an optimisation instance so that low cost solutions become high probability points in the learned distribution. At inference, the model generates multiple candidate vectors by iterative denoising and selects the best, consistently surpassing mixed-integer programming and heuristic baselines on diverse benchmarks. Because training only requires a scalar cost evaluator, DiffSG is agnostic to the exact form of the objective and naturally accommodates multi objective or non-linear formulations, an attractive property for CNF/SFC placement where latency, cost and energy often interplay.

A complementary line of research embeds diffusion models inside reinforcement learning (RL) pipelines. Deep Diffusion Soft Actor-Critic (D2SAC) [14] augments the actor of Soft Actor-Critic with a DDPM that proposes multimodal action sequences. The richer policy distribution accelerates convergence and improves final rewards on standard control benchmarks, implying that similar hybrids could enhance RL-based network orchestration by sampling diverse placement schedules that simple Gaussian policies cannot represent.

Because many network embedding problems are graph structured, several studies pair diffusion processes with GNNs. GDSG [13] tackles multi-hop computation off-loading by placing a GNN-based denoiser atop a Gaussian diffusion chain; trained solely on heuristic solutions, the model recovers near-optimal task assignments with high probability. DI-FUSCO [10] generalises this idea to canonical NP-hard problems such as the Travelling Salesman Problem and Maximum

Independent Set, cutting the optimality gap of previous neural solvers by roughly 50% on graphs with thousands of nodes. These successes indicate that graph conditioned diffusion can efficiently explore huge discrete spaces, a characteristic shared with CNF placement.

Beyond diffusion, learning-based methods continue to advance state of the art placement in novel environments. For Low Earth Orbit (LEO) satellite constellations, Doan et al. [17] combine multi-agent Q-learning for chain scheduling with Bayesian optimisation for Virtual Network Function (VNF) caching. Although diffusion models have not yet been applied in LEO scenarios, their ability to sample global, constraint aware configurations suggests a promising future direction.

Very recently, Network Diffuser [15] applies conditional diffusion directly to online SFC placement and scheduling. The authors view the problem as generating a feasible state trajectory: a DDPM denoises random sequences of placements into valid ones while conditioning on resource limits and latency requirements. Lacking expert traces, they create synthetic training data via inverse optimisation. It demonstrates the practicality of tailoring diffusion to complex SFC placement tasks.

In summary, existing work shows that diffusion models can learn expressive, conditioned distributions over feasible solutions, yielding sub-optimal performance across routing, off-loading and scheduling domains. Nevertheless, no prior study has addressed CNF placement across the Cloud-Continuum. The present paper bridges this gap by designing a topology-aware, GNN-based DDPM that generates full CNF mappings on the clouds available. Table I contrasts diffusion-based optimizers most relevant to our study with classical VNE/SFC lines. We differ from previous works by targeting CNF placement across the Cloud-Continuum with strict constraints, solved via a graph-conditioned DDPM that outputs full CNF→cloud assignments and is evaluated against an exact MINLP under a fixed time budget.

III. SYSTEM MODEL

A. Network model

The system model described in Figure 1 and formalized in Table II is used for formalizing the problem of CNF placement under constraints. We assume that the system consists of multiple clouds $\mathcal{C} = \{1, 2, \dots, C\}$ where the computational tasks of an atomic CNF are placed, instantiated and executed. The clouds in $\mathcal{C}$ are independently managed and operated, but

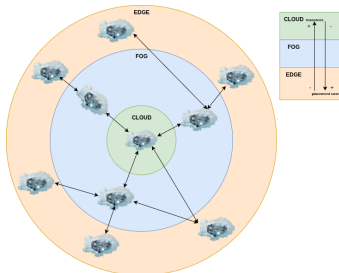


Fig. 1. Network model of the problem.

we assume that they are connected in an arbitrary graph $\mathcal{G}_C$ such that at least one path exists from cloud i to cloud j , for every $i, j \in \mathcal{C}$. The set of neighbours of cloud i is denoted as $\mathcal{N}_i$, for any $i \in \mathcal{C}$. We model the connections between two clouds, for simplicity, as a single link with capacity c_{ij} bits/s.

The graph $\mathcal{G}_C$ is static and predetermined, i.e., it is assumed that the communication network among the clouds does not change over the lifetime of the system. This corresponds to a static view of the scheduling and placement problem, in which the long-term performance goals have a well-defined steady state that, when reached, does no change with time.

B. SFCs and CNFs

For the purposes of this work, a SFC s_h , $h = 1, \dots, H$, is a partially ordered sequence of atomic computing tasks, i.e., each constituent computing task cannot be split or divided further into smaller, separable tasks. This individual, non-decomposable tasks are called CNFs, and we assume without loss of generality that the CNFs can be taken from a finite set of M possible CNFs, $\mathcal{F} = \{f_1, \dots, f_M\}$. An example of CNFs and SFCs can be seen in Fig. 2

An SFC s_h is modelled as a directed acyclic graph (DAG) $\mathcal{S}_h$ with one starting node and one terminating node. In other words, $\mathcal{S}_h$ has a root node, a sink node, and a number of intermediate nodes, with the condition that there are no cycles in $\mathcal{S}_h$. In the DAG $\mathcal{S}_h$, node $u \in \mathcal{F}$ precedes node $v \in \mathcal{F}$ if there is an edge $u \rightarrow v$. The DAG has a natural *topological ordering* of its nodes, $\text{ord}(\mathcal{S}_h) = (s_h^{(1)}, s_h^{(2)}, \dots, s_h^{(\ell_h)})$, wherein if $u \rightarrow v$, then u appears before v in the list $\text{ord}(\mathcal{S}_h)$.

We further assume that each individual CNF $s_h^{(j)}$ in a SFC $\mathcal{S}_h$ can be placed in a cloud from $\mathcal{C}$, but needs some amount of computing resources $\mathbf{z}_h^{(j)} = (x_h^{(j)}, d_h^{(j)}, \mathbf{r}_h^{(j)})$ where $x_h^{(j)}$ is the amount of computing resource (e.g. number of CPUs), $d_h^{(j)}$ is the amount of storage units (e.g. amount of RAM), and $\mathbf{r}_h^{(j)}$ is the vector of communication rates from CNF $s_h^{(j)}$ to each of its subsequent CNFs in the SFC. To account for possible restrictions on the usage of resources, we also assume that a cloud $i \in \mathcal{C}$ can only execute a predefined set of CNF types $\mathcal{T}_i \subseteq \mathcal{F}$. Thus, a CNF $s_h^{(j)}$ can only be placed at cloud i if $s_h^{(j)} \in \mathcal{T}_i$. Obviously, the case $\mathcal{T}_i = \mathcal{F}$ means that any cloud can host an arbitrary CNF of any type. Additionally, observe that in our model the resource vector $\mathbf{z}_h^{(j)}$ is only dependent on the type of the CNF, regardless its placement in the available clouds.

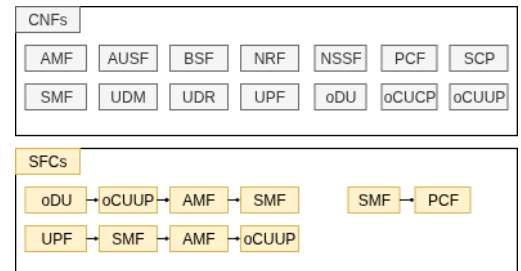


Fig. 2. Example of CNFs and SFCs

TABLE I
COMPARISON WITH STATE OF THE ART SOLUTIONS

Work	Problem focus	Output type	Conditioning / data	Constraint handling
DiffSG [11]	Generic network optimization	Continuous vectors, with discretization for binary	Black-box cost evaluator; instance-conditioned diffusion; no optimal labels needed	Constraints folded into objective via penalties; sampling selects low-cost solutions
GDSG [13]	MEC multi-hop computation offloading; discrete offload + continuous alloc.	Hybrid discrete + continuous heads	GNN-conditioned diffusion; sub-optimal heuristic data suffices; task orthogonality between heads	Feasibility promoted via training loss design; final selection over samples
Network Diffuser [15]	Online SFC placing & scheduling	Trajectory/state sequences	Conditional diffusion over state rollouts; inverse demonstrations	Feasibility via conditioning and guidance during denoising
DIFUSCO [10]	Classic CO on graphs (TSP, MIS, etc.)	Discrete (binary/perm.)	Graph diffusion (Gaussian/Bernoulli) on node/edge vars	Validity encouraged by process design + schedules
This work	CNF/SFC placement on Cloud-Continuum with restrictions	Discrete CNF→cloud assignment	Heterogeneous graph (cloud+CNF) conditions a DDPM; no labels (noise MSE) + constraint losses; best-of- k	Hard constraints enforced by mask + penalties; choose best feasible sample

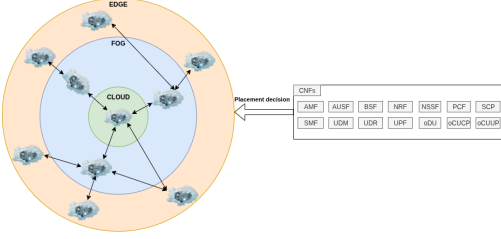


Fig. 3. CNF placement problem on the Cloud-Continuum

In our system model, a fixed collection of SFCs $\mathcal{S} = \{s_1, s_2, \dots, s_H\}$ is known in advance. This collection must be placed in the system without violating the resource constraints, the partial topological ordering of the constituent CNFs for each SFC, while optimising a cost function described below.

C. Problem formulation

We will assume that the placement of a CNF $s_h^{(j)}$ at cloud i has a fixed cost t_{im} , where $m = s_h^{(j)}$ is the type of the CNF. The execution of a computing task at the CNF has a fixed delay τ_m , and the delay for transmitting a message from a CNF running in cloud i to a CNF running in cloud j is equal to κ/c_{ij} , where κ is a suitable but arbitrary constant that can

TABLE II
SYSTEM MODEL NOTATION

Symbol	Meaning
$\mathcal{C}, C$	Set / number of clouds
$\mathcal{F}, M$	Set / number of CNFs
$\mathcal{S}, H$	Set / number of service function chains
X_i, D_i	CPU / RAM capacity of cloud i
x_m, d_m	CPU / RAM demand of CNF m
$t_{i,m}$	Cost of placing CNF m at cloud i
$c_{i,j}$	Bandwidth capacity on link (i, j)
τ_m	Processing delay of CNF m
$r_{u,v}$	Rate demand for hop $(u \rightarrow v)$
τ_h	End-to-end delay budget of chain h

be interpreted as the size (in bits) of the message sent from i to j , supposed of constant length here.

Let $x_{ijh} = 1$ if CNF $s_h^{(j)}$ is placed at cloud i , and $x_{ijh} = 0$ otherwise. The problem of optimal placement (Fig. 3) of the collection of SFCs $\mathcal{S}$ on the cloud federation $\mathcal{C}$ with static topology $\mathcal{G}_C$ can be formulated as follows

$$\min \sum_{h=1}^H x_{ijh} t_{im} \quad (1)$$

$$\sum_{i \in \mathcal{C}} x_{ijh} = 1, \quad \forall h \in [H], j \in [\ell_h] \quad (1a)$$

$$x_{ijh} e_{jj'}^{(h)} x_{i'j'h} \leq e_{ii'}, \forall i, i' \in \mathcal{C}, h \in [H], j \neq j' \in [\ell_h] \quad (1b)$$

$$\sum_{h=1}^H x_{ijh} x_h^{(j)} \leq X_i, \quad \forall i \in \mathcal{C} \quad (1c)$$

$$\sum_{h=1}^H x_{ijh} d_h^{(j)} \leq D_i, \quad \forall i \in \mathcal{C} \quad (1d)$$

$$\sum_{h=1}^H x_{ijh} e_{jj'}^{(h)} x_{i'j'h} r_h^{(jj')} \leq c_{ii'}, \quad \forall i \in \mathcal{C}, j, j' \in [\ell_h] \quad (1e)$$

$$J(\mathbf{x}_h, \mathcal{G}_C) \leq \tau_h, \quad \forall h \in [H] \quad (1f)$$

$$J(\mathbf{x}_h, \mathcal{G}_C) = \max_{p \in \mathcal{P}_h} \left\{ \sum_{i=1}^C \sum_{q=1}^{h_p-1} \left(\tau_{ij_q} + \frac{\kappa}{c_{jqj_{q+1}}} \right) x_{ij_q h} + \tau_{ij_{h_p}} x_{ij_{h_p} h} \right\} \quad (1g)$$

$$x_{ijh} = 0, \quad \forall i \in \mathcal{C}, \forall h \in [H], \quad (1h)$$

$$\forall j \in [\ell_h] \text{ s.t. } s_h^{(j)} \notin \mathcal{T}_i \quad (1i)$$

$$x_{ijh} \in \{0, 1\} \quad \forall i \in \mathcal{C}, h \in [H], j \in [\ell_h] \quad (1i)$$

These equations have the following interpretation:

- (1) is the cost function of the placement, extended to all the SFCs.
- Constraints (1a) enforces that each CNF $s_h^{(j)}$ is placed in one and only one cloud i .
- Constraints (1b) are the topological restrictions: the CNFs from a SFC s_h must be placed such that a physical link exists between the placement i of CNF $s_h^{(j)}$ and

the placement i' of CNF $s_h^{(j')}$ if $s_h^{(j)}$ precedes $s_h^{(j')}$. The variables $e_{ii'}$ are binary and equal 1 if and only if the clouds i and i' are directly connected. Similarly, the variables $e_{jj'}$ are equal to 1 if and only if $s_h^{(j)}$ precedes $s_h^{(j')}$. Consecutive CNFs may be co-located in the same cloud, i.e., we assume $e_{ii} = 1$ so that zero-hop “links” are allowed.

- Constraints (1c) and (1d) are the constraints on the physical resources at cloud i .
- (1e) are the constraints on the communication rate between neighbouring clouds i, i' : the sum of requested rates from all the CNFs placed at i towards the CNFs placed at i' cannot exceed the available bandwidth $c_{ii'}$.
- (1f) are the delay constraints, where τ_h is the maximum allowed delay for completing SFC s_h . Here, (1g) denotes the delay for the placement $\mathbf{x}_h := (x_{ijh} : i \in \mathcal{C}, j \in [\ell_h])$ on $\mathcal{G}_C$. On the DAG that defines the SFC s_h and the placement $\mathbf{x}_h$, this delay can be computed with the following expression: let $p = (j_1, j_2, \dots, j_{h_p})$ be a path with h_p nodes through the DAG of the SFC s_h , where every index $j_q \in [\ell_h]$, i_1 is the root node and j_{h_p} is the sink node. Then, the delay for completing this SC is the maximum delay along any path for SCF, given the placement $\mathbf{x}_h$.
- Constraint (1h) encodes CNF-to-cloud compatibility: a CNF of type $s_h^{(j)}$ may be placed at cloud i only if $s_h^{(j)} \in \mathcal{T}_i$. The unconstrained case corresponds to $\mathcal{T}_i = \mathcal{F}$.
- (1i) simply states that the placement decision variable is binary.

There exist some considerations to be taken into account on the above model:

- 1) We have assumed that CNFs are unique but a single CNF placed at a cloud is not dedicated to a single service function chain, i.e., a particular CNF can be shared by multiple SFCs or multiple requests.
- 2) In our mathematical model, we have not allowed the possibility that a cloud forwards a message to another cloud hosting the next CNF to be executed. In other words, we require that, if for a SFC s_h , the CNF $s_h^{(j)}$ precedes $s_h^{(j')}$, and they are placed at clouds i and i' , respectively, then there is a direct link between clouds i and i' .
- 3) Observe that the constraints (1b) and (1e) can be linearized, but the delay function (1f) is non-linear. The only case where it is linear is when the DAG is actually a line, i.e., the SFC is a sequential chain of CNFs.

IV. DIFFUSION-BASED SOLVER

In this Section, we present a diffusion-based framework for the CNF placement problem (see Figure 4). Unlike classical integer-programming solvers, our approach learns to generate feasible low-cost assignments by treating placement as a generative task. We adapt the DDPM framework to this discrete combinatorial setting by using a GNN to encode the network topology, the CNF structure and by training with additional loss terms that enforce placement constraints.

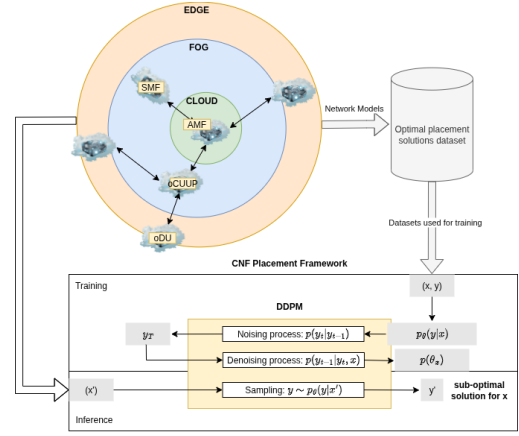


Fig. 4. Diffusion-based CNF placement framework architecture

Specifically, we design a neural network architecture that takes as input a heterogeneous graph representing the network and CNFs, along with an assignment matrix. The model embeds node features via message-passing, incorporates the diffusion timestep, and outputs per-(CNF, cloud) scores interpreted as predicted noise. During training, we add Gaussian noise to the CNF-to-cloud assignment matrix and minimize the MSE between the model’s predicted noise and the true noise, following the DDPM paradigm. Crucially, we also integrate specific loss terms: penalties for capacity violations, adjacency violations, bandwidth overuse, SFC delay budget violations and placement-restriction violations, following the equations presented in Sec. III-C. At inference time, we run the reverse diffusion process to sample many candidate assignments and pick the best valid solution.

A. Graph-based Diffusion Architecture

1) *Graph Representation*: To apply a diffusion model, we first represent the CNF placement instance as a graph structured input. We use a heterogeneous graph H with two node types: cloud nodes and CNF nodes.

- *Cloud nodes*: Each cloud i has a feature vector $[X_i, D_i, t_i]$ for its CPU, RAM and placement cost. We maintain a tensor of cloud type indices which is embedded into a learned type embedding. Cloud-Cloud edges encode the network connectivity: if bandwidth $c_{ij} > 0$, we include a directed edge $(i \rightarrow j)$ with attribute $[c_{ij}]$.
- *CNF nodes*: Each CNF f has a feature vector $[x_f, d_f, \tau_m]$ (CPU, RAM, processing delay). We have a restriction cloud placement index embedded into a learned vector. Within each SFC, we add edges between consecutive CNFs: for chain edges $s_h^j \rightarrow s_h^{j+1}$, we include an edge with attributes $[\ell_{j,j+1}, \Delta_{s_h}, h, k]$ meaning the communication rate between consecutive CNFs, the SFC delay, an SFC identifier and the hop index in the SFC.

Finally, we build a complete bipartite graph from each CNF to each allowed cloud node respecting placement restrictions. This heterogeneous graph captures all data: node features encode capacities and requirements; and edges encode topology: physical links and service chains.

2) *GNN Architecture*: The backbone model for propagating and aggregating information over H is a GNN that predicts the added noise on the assignment matrix. It implements a conditional DDPM where the condition is the graph H . We include learnable embeddings for cloud type and for CNF restriction, and these are concatenated with the raw node features so that the model knows node types and any CNF placement restriction. Also, the diffusion timestep or noise level $\log \sigma_t$ is mapped through a two-layer multilayer perceptron (MLP) to produce a hidden embedding t_{emb} . This is then added to every node's embedding in each layer, allowing the GNN to be explicitly aware of the noise scale.

Two separate graph encoders process the cloud and CNF subgraphs. For clouds, we have an encoder with multiple GraphSAGE convolution layers augmented with edge attributes. Similarly, an encoder for CNF with GraphSAGE layers processes the chain subgraph. Formally, let x_c and x_t be the augmented cloud and CNF feature matrices. The cloud encoder performs

$$h_c = \text{CloudEncoder}(x_c, E_{cc}, A_{cc}) \quad (2)$$

where E_{cc} and A_{cc} are the cloud-cloud edge indices and bandwidth attributes. Likewise, the CNF encoder outputs

$$h_t = \text{CNFEncoder}(x_t, E_{tt}, A_{tt}) \quad (3)$$

for CNF–CNF edges. Each of these embeddings has dimension H . After encoding, we add the same time embedding t_{emb} to all node vectors.

The network must reason about which CNF can go to which cloud. To mix information between cloud and CNF embeddings, we apply two GraphSAGE layers on the bipartite assignment edges. Let E_{tc} denote the edge index of $CNF \rightarrow \text{cloud}$ assignment edges and E_{ct} the reverse. We compute

$$h'_c = h_c + \text{SAGEConv}_{t \rightarrow c}(h_t, h_c; E_{tc}), \quad (4a)$$

$$h'_t = h_t + \text{SAGEConv}_{c \rightarrow t}(h'_c, h_t; E_{ct}), \quad (4b)$$

The model ultimately needs to predict a score for each possible (CNF, cloud) pair. To do this, we form the Cartesian product of the updated embeddings h'_t and h'_c . Concretely, for M CNFs and C clouds, we create $M \times C$ pairs (h'_{t_i}, h'_{c_j}) and also include the current noisy assignment value $y_{i,j}$ as input. These vectors are fed through a decoder MLP with two hidden layers and an activation function, producing a scalar *score* for each pair. The output is reshaped to an $M \times C$ matrix S . Thus, the model outputs unnormalized scores $s_{i,m}$ for assigning CNF m to cloud i .

B. Training process

1) *Forward diffusion*: We use a standard Gaussian diffusion schedule with T timesteps and precompute a cosine schedule $\sqrt{\alpha_t}$ and σ_t for $t = 1, \dots, T$. The forward noising process adds scaled Gaussian noise to the true assignment matrix $Y_0 \in \{0, 1\}^{M \times C}$. At a random timestep t , we draw noise $E \sim \mathcal{N}(0, I)$, masked to valid placements, and set

$$Y_t = \sqrt{\alpha_t} Y_0 + \sigma_t E, \quad (5)$$

Then, it is applied the placement mask, where forbidden placements stay zero. This ensures Y_t is a noisy real-valued matrix with the same sparsity pattern. The training goal is for the model to predict the noise E .

2) *Train*: The diffusion model is trained to minimize the expected squared error between the predicted noise and the true noise. Concretely, the network outputs a real-valued noise prediction $\hat{e}$ of the same shape as the noisy assignment. The denoising objective follows the standard ϵ -prediction parametrization of DDPMs. Given a random timestep t , we diffuse the assignment Y_0 to obtain Y_t and the injected Gaussian noise E . The GNN predicts $\hat{E} = \epsilon_\theta(H, Y_t, \sigma_t)$, and we minimize the mean-squared error

$$\mathcal{L}_{\text{denoise}} = \mathbb{E}_t \left[\left\| \hat{E} - E \right\|_2^2 \right]. \quad (6)$$

From the predicted noise, we reconstruct the clean assignment estimate

$$\hat{Y}_0 = (Y_t - \sigma_t \hat{e}) / \sqrt{\alpha_t}. \quad (7)$$

Then, we forbid illegal placements and obtain the probability matrix that feeds every constraint loss

$$P = \text{Softmax}(\hat{Y}_0 \odot M - \infty \cdot (1 - M)) \quad (8)$$

where M is the placement mask. The full training procedure is summarised in Algorithm 1

3) *Constraint loss terms*: After predicting the noise $\hat{E}$, we reconstruct

$$\hat{Y}_0 = \frac{Y_t - \sigma_t \hat{E}}{\sqrt{\alpha_t}}, \quad (9)$$

apply the placement mask (forbidden CNF-cloud pairs), and obtain a relaxed assignment matrix $P \in [0, 1]^{M \times C}$ via a row-wise softmax:

$$P = \text{softmax}(\text{Mask}(\hat{Y}_0)), \quad \sum_{i=1}^C P_{m,i} = 1, \quad \forall m. \quad (10)$$

All constraint penalties are computed on P and added to the denoising objective.

Placement restrictions. Let $\text{mask}_{m,i} \in \{0, 1\}$ indicate whether CNF m is allowed on cloud i . Although masking is enforced before the softmax, we also penalize any residual probability mass assigned to forbidden pairs:

$$\mathcal{L}_{\text{restr}}(P) = \sum_{m=1}^M \sum_{i=1}^C \left((1 - \text{mask}_{m,i}) P_{m,i} \right)^p, \quad (11)$$

CPU/RAM capacity. Let x_m and d_m be CPU/RAM demands of CNF m , and X_i, D_i the CPU/RAM capacities of cloud i . The expected resource usage under P is

$$U_i^{\text{cpu}}(P) = \sum_{m=1}^M P_{m,i} x_m, \quad U_i^{\text{ram}}(P) = \sum_{m=1}^M P_{m,i} d_m \quad (12)$$

Algorithm 1 Training

Require: Dataset $\mathcal{D}$; heterogeneous graph builder $H(\cdot)$; mask M ; steps T ; schedule $\{\alpha_t\}_{t=1}^T$

- 1: **for** instances **do**
- 2: Build graph H and assignment Y_0
- 3: Sample $t \sim \mathcal{U}\{1..T\}$, $\epsilon \sim \mathcal{N}(0, I)$; set $Y_t = \sqrt{\alpha_t}Y_0 + \sqrt{1 - \alpha_t}\epsilon$
- 4: Predict $\hat{\epsilon} = \epsilon_\theta(H, Y_t, t)$
- 5: Reconstruct $\hat{Y}_0 = (Y_t - \sqrt{1 - \alpha_t}\hat{\epsilon})/\sqrt{\alpha_t}$
- 6: Apply placement mask: $\hat{Y}_0 \leftarrow \hat{Y}_0 \odot M + (-\infty) \cdot (1 - M)$
- 7: Row-wise probabilities $P = \text{softmax}(\hat{Y}_0)$
- 8: Compute losses: $\mathcal{L}_{\text{DDPM}} = \|\epsilon - \hat{\epsilon}\|_2^2$; plus $\sum_u \lambda_u \mathcal{L}_u(P; H)$
- 9: Update θ with Adam on $\mathcal{L}$
- 10: **end for**

We penalize capacity violation with a mean ReLU:

$$\mathcal{L}_{\text{cap}}(P) = \frac{1}{C} \sum_{i=1}^C \text{ReLU}(U_i^{\text{cpu}}(P) - X_i) + \frac{1}{C} \sum_{i=1}^C \text{ReLU}(U_i^{\text{ram}}(P) - D_i) \quad (13)$$

Row-sum regularizer. We include a soft regularizer that encourages each CNF row to sum to one:

$$\mathcal{L}_{\text{rowsum}}(P) = \frac{1}{M} \sum_{m=1}^M \left(\sum_{i=1}^C P_{m,i} - 1 \right)^2. \quad (14)$$

Note that, because P is obtained via a row-wise softmax, this term is typically close to zero and acts mainly as a numerical sanity check.

Adjacency constraint. Let $c \in \mathbb{R}_+^{C \times C}$ be the cloud-to-cloud bandwidth matrix, and define the adjacency indicator $A_{i,j} = \mathbb{I}[c_{i,j} > 0]$. For each SFC hop edge $(u \rightarrow v)$ in the CNF chain graph, define the outer-product flow tensor

$$F_{i,j}^{(u,v)}(P) = P_{u,i} P_{v,j}. \quad (15)$$

The adjacency penalty is the average probability mass assigned to non-existent links:

$$\mathcal{L}_{\text{adj}}(P) = \frac{1}{|\mathcal{E}_{\text{sfc}}| C^2} \sum_{(u,v) \in \mathcal{E}_{\text{sfc}}} \sum_{i=1}^C \sum_{j=1}^C F_{i,j}^{(u,v)}(P) (1 - A_{i,j}). \quad (16)$$

Bandwidth constraint. Let $r_{u,v}$ be the communication rate (bps) required by hop $(u \rightarrow v)$. The expected aggregate demand on physical link (i, j) is

$$L_{i,j}(P) = \sum_{(u,v) \in \mathcal{E}_{\text{sfc}}} r_{u,v} F_{i,j}^{(u,v)}(P). \quad (17)$$

We penalize any overuse with

$$\mathcal{L}_{\text{bw}}(P) = \frac{1}{C^2} \sum_{i=1}^C \sum_{j=1}^C \text{ReLU}(L_{i,j}(P) - c_{i,j}). \quad (18)$$

End-to-end delay constraint For each hop $(u \rightarrow v)$, let τ_v be the processing delay of CNF v . We use a differentiable surrogate that distinguishes co-located and cross-node execution.

Let $\mathbb{I}[i = j]$ denote co-location and define a fixed message size κ (in bits) used to model cross-node propagation as $\kappa/c_{i,j}$. The expected per-hop delay is

$$\mathbb{E}[d_{u,v}](P) = \sum_{i=1}^C \sum_{j=1}^C F_{i,j}^{(u,v)}(P) \left(\mathbb{I}[i = j] \cdot \tau_v + \mathbb{I}[i \neq j] \cdot (\tau_v + \kappa/c_{i,j}) \right). \quad (19)$$

We sum these expected hop delays per service chain h and add the processing delay of the last CNF once more, obtaining $D_h(P)$. The penalty is then

$$\mathcal{L}_{\text{delay}}(P) = \frac{1}{H} \sum_{h=1}^H \text{ReLU}(D_h(P) - \tau_h), \quad (20)$$

where τ_h is the delay budget of chain h .

Finally, the total training objective is

$$\mathcal{L} = \mathcal{L}_{\text{denoise}} + \lambda_{\text{cap}} \mathcal{L}_{\text{cap}} + \lambda_{\text{restr}} \mathcal{L}_{\text{restr}} + \lambda_{\text{rowsum}} \mathcal{L}_{\text{rowsum}} + \lambda_{\text{adj}} \mathcal{L}_{\text{adj}} + \lambda_{\text{bw}} \mathcal{L}_{\text{bw}} + \lambda_{\text{delay}} \mathcal{L}_{\text{delay}}. \quad (21)$$

Because all constraints act on P , our score network remains an unbiased estimator of the Gaussian noise while feasibility is enforced entirely in data space. The training and constraint losses procedure is summarized in Algorithm 1.

C. Inference process

After training, we use the learned GNN backbone to sample placements for new instances. The sampling procedure is the standard reverse diffusion (DDPM) schedule. We start from pure noise $Y_T \sim \mathcal{N}(0, I)$ of shape $1 \times M \times C$ scaled by σ_T , and, for $t = T - 1, \dots, 0$, we iteratively predict and remove noise:

- 1) Given current Y_t and noise level σ_t , compute $\hat{E} = \text{GNN}(H, Y_t, \sigma_t)$.
- 2) Estimate the underlying assignment $\hat{Y}_0 = (Y_t - \sigma_t \hat{E})/\sqrt{\alpha_t}$.
- 3) If $t > 0$, compute the mean of Y_{t-1} as $\mu = \sqrt{\alpha_{t-1}} \hat{Y}_0$. After this compute, sample $Y_{t-1} = \mu + \sigma_{t-1} \zeta$, where ζ is fresh Gaussian noise.

After reaching $t = 0$, we have a denoised matrix $\hat{Y}_0$. We then apply the placement mask: forbidden entries of $\hat{Y}_0$ are set to $-\infty$, and finally we take a Softmax across clouds for each CNF to produce assignment probabilities. This yields a matrix of probabilities $p_{m,i}$, where the highest-probability cloud is chosen for each CNF to form a discrete placement. In practice, we generate many samples and pick the one with lowest cost that satisfies all constraints. The best-of- k sampling and selection used at inference is summarised in Algorithm 2.

D. Complexity of the solution

Let $|V|$ and $|E|$ be the numbers of nodes and edges in the heterogeneous graph H , and let L be the number of GNN layers. Each reverse step applies L message-passing layers whose cost is $O(|E|)$ (and $O(|V|)$ for node updates), hence one denoise step is $O(L \cdot (|E| + |V|))$. With T diffusion steps and k independent samples, the inference cost is

Algorithm 2 Inference

Require: Instance graph H ; mask M ; steps T ; schedule $\{\alpha_t\}$; samples k

- 1: **for** $s = 1$ to k **do**
- 2: Initialize $Y_T \sim \mathcal{N}(0, I)$
- 3: **for** $t = T, \dots, 1$ **do**
- 4: $\hat{\epsilon} = \epsilon_\theta(H, Y_t, t)$
- 5: $\hat{Y}_0 = (Y_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}) / \sqrt{\bar{\alpha}_t}$; mask $\hat{Y}_0 \leftarrow \hat{Y}_0 \odot M + (-\infty) \cdot (1 - M)$
- 6: Compute mean μ_t of $p_\theta(Y_{t-1} | Y_t)$; sample Y_{t-1}
- 7: **end for**
- 8: Discretize: $P = \text{softmax}(\hat{Y}_0)$; $\tilde{X} = \text{argmax}_{\text{row}}(P)$
- 9: Evaluate feasibility and cost of $\tilde{X}$
- 10: **end for**
- 11: Return best feasible $\tilde{X}$ among k ; if none feasible, return lowest-violation sample

$O(k \cdot T \cdot L \cdot (|E| + |V|))$, which grows roughly linearly with graph size and linearly in T . In contrast, MINLP solve time grows super-linearly and often exponentially with instance size due to branching and nonlinear constraints, leading to frequent timeouts on moderate/large cases. Our empirical results (Sec. V) reflect this gap: diffusion runtime scales gently with $|C|$ while MINLP times increase steeply, with several instances hitting the time limit.

V. RESULTS

A. Evaluation

The evaluation of the diffusion-based solver was conducted by generating 44 network model scenarios, each characterised by distinct parameters, including the number of nodes and their respective resources, as well as the number of SFCs of varying lengths. The MINLP-based solver, created using GEKKO, is utilised in order to ascertain optimal solutions for the network models. The configuration is designed to identify the minimum-cost placement that is constrained-compliant. The system solves 39 scenarios, of which 5 scenarios extend beyond the maximum time allowance of 4 hours without yielding an optimal solution. The diffusion model, conversely, is utilised to generate 50 solution samples for each instance, from which the optimal solution that fulfils all the constraints is selected.

For training and evaluating the diffusion-based solver, we split the 44 network model scenarios in two datasets:

- **Train dataset:** formed by 20 network model scenarios, all with optimal solutions from the MINLP solver.
- **Evaluation dataset:** formed by 24 network model scenarios different from the train dataset, including the 5 scenarios where the MINLP solver has not found any optimal solution.

With this set, we evaluate the diffusion-based solver and compare its performance to the MINLP-based solver in terms of solution feasibility, quality and efficiency. Our experiments are designed to answer the following questions:

- 1) Does the diffusion model reliably generate feasible placements that meet all constraints?
- 2) How close are the diffusion model's solutions to optimal in terms of total deployment cost?

- 3) What is the runtime advantage of the learned generative solver, especially as problem size scales?
- 4) Can the diffusion model generalize to scenarios that were not seen during training?

In comparison with the previous version of this work, the evaluation in this paper is significantly extended in two directions. Firstly, the following additional baselines are introduced: two greedy heuristics (denoted Heuristic A and Heuristic B) and several learning-based solvers based on GDSG (Diffusion-GNN, Diffusion-MLP and a supervised GNN). Secondly, the definition of three additional evaluation families beyond the original validation set is proposed, with a view to assessing robustness and generalisation.

- *eval_big*: larger instances with more nodes, CNFs and SFCs than those used for training and validation.
- *eval_shift*: instances of similar size to the validation set but with shifted placement costs and parameters, breaking the original Cloud/Fog/Edge cost structure and stressing cost generalization.
- *eval_change*: a constraint-tight regime where effective message size and link capacities are modified so that bandwidth and delay constraints become strongly binding. This regime is designed to better reflect scenarios where network resources are scarce and end-to-end SFC latency is critical.

In all cases, the same system model and feasibility checker is employed for all methods, thereby ensuring that every solver (MINLP, diffusion, heuristics and learning baselines) is evaluated under precisely the same constraints and cost function.

B. Metrics

We use the following key metrics to compare methods:

- *Feasibility Rate*: The percentage of instances for which a method returns a solution that is both practical and achievable. It is an irrefutable fact that, by definition, whenever a solution is returned by the MINLP solver, it is always feasible. However, in the event that the solver fails to find a solution within the stipulated time limit, this is taken as evidence that the solver is not effectively feasible.
- *Total Deployment Cost*: The objective value of a placement that is feasible. In the validation set, the cost of each solver's placements is compared to the optimal cost from the MINLP. The present study reports both the average cost of the solutions and, when applicable, the relative optimality gap with respect to GEKKO, as defined as follows:

$$\text{gap} = \frac{\text{cost} - \text{cost}_{\text{GEKKO}}}{\text{cost}_{\text{GEKKO}}},$$

averaged over instances where both the method and GEKKO return feasible solutions.

- *Inference Time*: The temporal requirement for the production of a solution. In the context of the diffusion model, the process of inference entails the repeated execution of the GNN denoiser, $T = 100$, on the specified instance, commencing from the initial state of Gaussian noise. The

complexity of the inference process is largely independent of the complexity of the optimisation problem, with the exception of a linear relationship with the size of the input graph. In contradistinction, the MINLP solver's runtime exhibits a marked increase with problem size, approaching exponential growth in practice.

- *Regret and Best-share*: In the context of the extended evaluation families, where the GEKKO method is not utilised due to its computational expense, a comparative analysis is conducted between methodologies and the optimal cost among all approximate solvers for each individual instance. For the purpose of this study, let c_i denote the cost achieved by method m (if feasible), and let c_i^* denote the cost achieved by the best method. We define the regret of method m on that instance as

$$r_i^m = \frac{c_i^m - c_i^*}{c_i^*},$$

and report the average regret over instances where m is feasible and c_i^* is defined. In addition, the *best-share* of each method is reported. By this, it is meant the fraction of instances where the method attains the best cost (within a small numerical tolerance) among all methods.

The utilisation of these metrics enables the characterisation of two distinct aspects. Firstly, the feasibility of solutions that are discovered by a given method can be ascertained. Secondly, the proximity of these solutions to optimality (or to the best available baseline) can be determined. Furthermore, the computational cost of these solutions can be assessed.

C. Results

Figure 5 summarizes the performance of the diffusion-based solver vs. the MINLP solver on representative test scenarios from the evaluation set.

In the first instance, the focus is on these results, and the analysis with our previous results [18] is followed. The comparison is made of both solvers, with consideration given only to the evaluation dataset and the scenarios for which the MINLP solver finds an optimal solution. In [18], the scope of this figure was limited to a subset of 19 instances. In this expanded study, the focus has been broadened to encompass all 39 GEKKO-solvable instances. In summary, the diffusion-based solver successfully identifies feasible solutions in 92.3% of the instances examined, thereby substantiating the efficacy of the model in identifying suboptimal solutions for network model scenarios, both those encountered during the training process and those that were not part of the training dataset. In terms of financial expenditure, the diffusion solver's best-found solution has an average cost of approximately 370 units

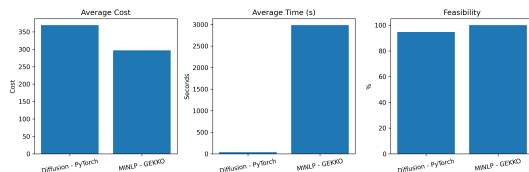


Fig. 5. Comparison between MINLP and diffusion solver.

for solvable instances, in comparison to an average optimal cost of approximately 298 units for the MINLP solver. This corresponds to an average optimality gap of approximately 25.6% units. It is acknowledged that a key benefit of the diffusion model is its ability to generate a comprehensive distribution of solutions. This characteristic enables the trade-off between quality and runtime by producing multiple samples. In instances where the initial sample does not closely resemble the optimal solution, a modest number of samples often yields at least one solution that significantly surpasses the mean quality.

In terms of runtime, the results of the study demonstrate that the diffusion approach is significantly more efficient. On the GEKKO-solvable set, the MINLP solver requires an average of approximately 1957 seconds per instance, whereas the diffusion solver produces 50 samples in approximately 29.5 seconds and can stop earlier if a good solution is found. As will be demonstrated in the following per-size analysis, this behaviour is also reflected therein.

In the instances where the MINLP solver encounters a timeout without yielding a solution, the diffusion-based solver identifies a suboptimal solution for all five scenarios. The metrics for these solutions are as follows: an average cost of 375 units and an average time of 53 seconds. This lends support to one of the key advantages of the diffusion-based solver over the MINLP solver: namely, its ability to identify suboptimal solutions for models with which the MINLP solver has been unable to find an optimal solution within the stipulated time limit.

Finally, the time taken to find a solution for each solver was compared as a function of the number of nodes in the network model scenarios. The time was then averaged over scenarios with the same number of nodes. As demonstrated in Fig. 6, the mean execution time for the MINLP solver exhibits a marked increase with the number of nodes, reaching a maximum of 14,400 seconds in instances that exceed the designated time limit. Conversely, as illustrated in Fig. 7, it is evident that the mean duration for the diffusion-based solver remains relatively constant as the number of nodes increases. This observation validates the hypothesis that the solver's execution time remains largely unaffected by the combinatorial intricacy of the optimization problem, exhibiting an approximate linear relationship with the graph size.

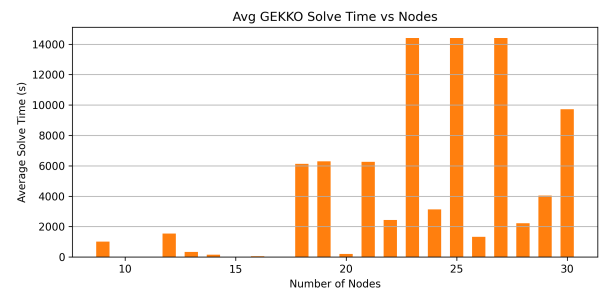


Fig. 6. Average time for MINLP solver vs. number of nodes.

TABLE III
OVERALL COMPARISON ON THE BASE BENCHMARK (44 SCENARIOS)

Method	GEKKO-solvable set (N=39)				GEKKO-timeout set (N=5)		
	Feas. (%)	Cost	Gap (%)	Time (s)	Feas. (%)	Cost	Time (s)
MINLP-GEKKO	100.0	298.46	0.00	1956.756	0.0	–	14400.000
(own) Diffusion-PyTorch	92.3	369.86	25.60	29.487	100.0	375.00	53.000
Diffusion-GNN (GDSG)	2.6	210.00	0.00	37.238	0.0	–	45.774
Diffusion-MLP (GDSG)	7.7	220.00	0.00	3.253	0.0	–	3.103
Supervised GNN (GDSG)	2.6	210.00	0.00	0.009	0.0	–	0.012
Heuristic A (SFC-centric)	71.8	329.29	13.10	0.024	60.0	361.67	0.090
Heuristic B (node+BW greedy)	92.3	298.06	0.20	0.011	100.0	319.00	0.024

D. Extended baseline comparison

The extended evaluation comprises supplementary baselines that extend beyond the MINLP and diffusion solvers. In this study, the focus is on two greedy heuristics (Heuristic A and Heuristic B) and three learning-based solvers derived from the GDSG framework. These solvers include an unsupervised Diffusion-GNN, an unsupervised Diffusion-MLP, and a supervised GNN. Table III summarizes the aggregate performance on the original 44-scenario benchmark, separating GEKKO-solvable instances from GEKKO timeouts.

The set under consideration is designated as the GEKKO-solvable set. In 39 out of the total number of instances, the MINLP solver achieves 100% feasibility and 0% gap by construction, with an average solve time of approximately 1956.8 seconds per instance. The diffusion-based solver achieves a feasibility rate of 92.3% on these instances, with an average cost of 369.9 units and an average optimality gap of 25.6%.

The two heuristics demonstrate markedly divergent behavioural patterns. It is evident that Heuristic A, which is SFC-centric, identifies feasible solutions for 71.8% of the GEKKO-solvable instances. The average cost of these solutions is 329.3 units, while the average gap is 13.1%. It is noteworthy that these processes exhibit negligible runtime, with an average of 24 milliseconds per instance. In contrast, Heuristic B, which is node- and bandwidth-aware, exhibits a combination of high feasibility (92.3% match with diffusion on this set) with an average cost of 298.1 units and an average gap of only 0.2%. Furthermore, it operates with a processing time of approximately 11 milliseconds per instance. In summary, the Heuristic B is, in essence, optimal with respect to the GEKKO benchmark. Furthermore, it is more than three orders of magnitude faster than MINLP and approximately three

orders of magnitude faster than diffusion.

It is evident that the learning baselines are non-competitive as solvers. The unsupervised Diffusion-GNN and the supervised GNN are only feasible on 2.6% of the GEKKO-solvable instances (one instance out of 39), while the Diffusion-MLP reaches 7.7% feasibility (three instances). In the limited number of instances where they are viable, these solutions correspond to the GEKKO optimum yielding an average discrepancy of 0%, yet they consistently fall short in producing valid solutions for the majority of cases. As a result, they cannot be regarded as practical solvers for this problem.

In the context of the five GEKKO time-limited instances, the observed behaviour aligns with the previous version [18]. This indicates that both the heuristics and the diffusion solver possess the capability to swiftly generate feasible solutions, while GEKKO reaches the 4 h limit. It is evident that Heuristic B attains 100th feasibility with an average cost of 319 units and 24 ms runtime, whereas the diffusion solver attains 100th feasibility with an average cost of 375 units and 53s runtime.

Difficulty-bucket analysis.

In order to facilitate a more profound comprehension of the manner in which performance varies in accordance with problem difficulty, a stratification of the 39 GEKKO-solvable instances has been undertaken, resulting in the establishment of three difficulty buckets (easy, medium, hard) based on their respective GEKKO solve times, with 13 instances allocated to each bucket. The feasibility of each bucket, along with its average gap and average time, is computed. Table IV reports feasibility, optimality gap and average runtime across buckets, highlighting how constraint satisfaction and suboptimality evolve with instance difficulty.

In the instances classified as *easy*, Heuristic B has been shown to demonstrate near-optimal performance. It achieves 92.3% feasibility with an average gap of 0.22% and a runtime of 6 ms. Meanwhile, the diffusion solver attains 92.3% feasibility but experiences a 22.4% gap and requires approximately 22.9s per instance. As demonstrated in Figure 1, the gap in diffusion increases to 25.0% for medium instances, with a feasibility rate of 84.6%. In contrast, Heuristic B maintains a feasibility rate of 84.6% and a gap of 0%, aligning with the performance of GEKKO on all medium instances that are feasible.

In the cases of greater complexity, diffusion still attains 100% feasibility, but with a 29.0% gap and an approximate runtime of 40.6 seconds. Conversely, Heuristic B attains 100%

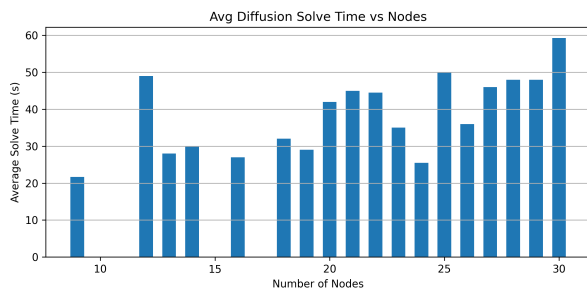


Fig. 7. Average time for diffusion-based solver vs. number of nodes.

TABLE IV
FEASIBILITY, OPTIMALITY GAP, AND RUNTIME BY DIFFICULTY BUCKET ON THE GEKKO-SOLVABLE SET (N=39)

Method	Easy (N=13)			Medium (N=13)			Hard (N=13)		
	Feas. (%)	Gap (%)	Time (s)	Feas. (%)	Gap (%)	Time (s)	Feas. (%)	Gap (%)	Time (s)
MINLP-GEKKO	100.0	0.00	585.900	100.0	0.00	1608.742	100.0	0.00	3675.627
Diffusion-PyTorch	92.3	22.40	22.923	84.6	25.02	24.923	100.0	28.99	40.615
Diffusion-GNN (GDSG)	0.0	–	29.653	7.7	0.00	30.021	0.0	–	52.040
Diffusion-MLP (GDSG)	0.0	–	3.748	7.7	0.00	3.014	15.4	0.00	2.997
Supervised GNN (GDSG)	0.0	–	0.008	7.7	0.00	0.010	0.0	–	0.010
Heuristic A (SFC-centric)	76.9	15.13	0.011	61.5	14.57	0.014	76.9	9.82	0.048
Heuristic B (node+BW greedy)	92.3	0.22	0.006	84.6	0.00	0.006	100.0	0.35	0.021

feasibility and a mere 0.35% gap in approximately 21 milliseconds. In terms of all the criteria under consideration, Heuristic B is shown to be the optimal solution, with a high level of efficiency. In contrast, the diffusion solver is found to be substantially suboptimal in terms of cost, and is significantly slower than the heuristics.

The learning baselines continue to demonstrate an almost complete lack of feasibility across all buckets (0–15% feasibility), thereby reinforcing the conclusion that they do not constitute effective solutions for this problem.

The ensuing results generalise the findings previously outlined in the previous paper [18]. With regard to the original instance distribution, a carefully engineered greedy heuristic (Heuristic B) is shown to be highly effective, essentially matching the MINLP optimum even on the most challenging instances. In contrast, the diffusion-based solver is found to be significantly suboptimal in terms of cost and considerably slower than the heuristics.

Robustness to scale and parameter shifts: eval_big and eval_shift

We next evaluate robustness under two types of distribution shift beyond the validation set considered in the previous version:

- **eval_big**: larger instances with more CNFs and SFCs than those used for training and validation.
- **eval_shift**: instances with the same size distribution as the validation set but with perturbed placement costs and parameters.

It is not possible to run GEKKO for these families due to the computational cost of solving many large instances. Consequently, the evaluation of methods is undertaken by means of relative regret and best-share with respect to the best cost achieved by any approximate solver on each instance. It is imperative to note that the statistics exclusively encompass instances wherein at least one solver successfully identifies a feasible solution. Table V reports feasibility, regret, and best-share on the three out-of-distribution families.

On **eval_big** ($N = 59$ instances with at least one feasible solution), Heuristic B clearly dominates, as can be seen in Fig. 8:

- Heuristic B achieves 100% feasibility, 0% average regret and best-share = 100%, with an average runtime of 30 ms.

- Heuristic A attains 86.4% feasibility, 6.29% average regret and best-share = 15.3%, with 0.323 s average runtime.
- Diffusion-PyTorch attains only 25.4% feasibility, with 31.61% average regret and best-share = 0%, requiring about 58.1 s per instance.

It is evident that, on larger graphs, the greedy heuristic B exhibits both robustness and near-optimality. Conversely, the diffusion solver’s feasibility is subject to collapse, and in instances of feasibility, its cost is significantly higher than that of the optimal solver.

On **eval_shift** ($N = 55$), where we modify cost and parameter distributions while keeping problem size similar to the validation set, the picture is similar, as can be seen in Fig. 9:

- Heuristic B attains 98.2% feasibility, 0% average regret and best-share = 98.2%, with about 29 ms runtime.
- Heuristic A reaches 80.0% feasibility, 27.24% average regret and best-share = 14.5%, with 0.107 s runtime.
- Diffusion-PyTorch maintains 98.2% feasibility, but suffers an average regret of 47.34% and best-share = 3.6%, with 27.6 s runtime.

The experiments conducted confirm the hypothesis that strong domain-specific heuristics can be extremely difficult to surpass. Heuristic B demonstrates excellent performance under

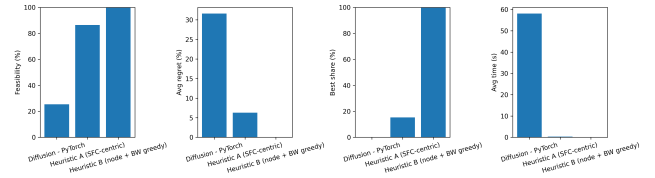


Fig. 8. Relative-cost evaluation on eval_big: feasibility, regret, best-share and runtime.

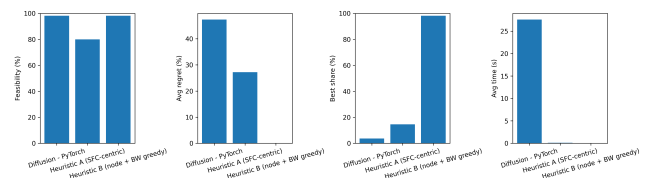


Fig. 9. Relative-cost evaluation on eval_shift: feasibility, regret, best-share and runtime.

TABLE V
RELATIVE-COST STATISTICS ON SCALE/PARAMETER-SHIFT AND CONSTRAINT-TIGHT FAMILIES.

Family	Method	Feas. (%)	Avg. regret (%)	Best-share (%)	Time (s)
eval_big (N=59)	Diffusion-PyTorch	25.4	31.61	0.0	58.068
eval_big (N=59)	Heuristic A (SFC-centric)	86.4	6.29	15.3	0.323
eval_big (N=59)	Heuristic B (node+BW greedy)	100.0	0.00	100.0	0.030
eval_shift (N=55)	Diffusion-PyTorch	98.2	47.34	3.6	27.582
eval_shift (N=55)	Heuristic A (SFC-centric)	80.0	27.24	14.5	0.107
eval_shift (N=55)	Heuristic B (node+BW greedy)	98.2	0.00	98.2	0.029
eval_change (N=35)	Diffusion-PyTorch	91.4	8.65	57.1	35.457
eval_change (N=35)	Heuristic A (SFC-centric)	17.1	0.00	17.1	0.041
eval_change (N=35)	Heuristic B (node+BW greedy)	31.4	0.18	28.6	0.014

both size and cost/parameter shifts, while the diffusion solver exhibits a significant degradation in performance.

Constraint-tight regime: eval_change

The most intriguing behaviour manifests in the eval_change family, wherein we deliberately constrain bandwidth and delay parameters to more accurately emulate a scenario characterised by limited network resources and the significance of end-to-end SFC latency. In practice, the effective message size employed for the calculation of per-hop network delay and link load is increased, whilst link capacities are reduced. This ensures that both bandwidth and delay constraints become strongly binding. It is precisely in this regime that local, greedy decisions are expected to become less reliable, and that global patterns learned from GEKKO solutions may prove advantageous. Table V quantifies the regime shift where feasibility becomes the dominant differentiator.

On **eval_change** ($N = 35$ instances with at least one feasible solution), the results are as follows, as can be seen in Fig. 10:

- Diffusion-PyTorch achieves 91.4% feasibility, with an average regret of 8.65%, best-share = 57.1% and average runtime of 35.5 s.
- Heuristic B achieves 31.4% feasibility, with an average regret of 0.18%, best-share = 28.6% and runtime 14 ms.
- Heuristic A achieves 17.1% feasibility, with 0% average regret and best-share = 17.1%, and average runtime 41 ms.
- All three learning-based solvers (Diffusion-GNN, Diffusion-MLP and supervised GNN) obtain 0% feasibility on these instances, despite consuming non-negligible runtime.

The following observations are of particular significance. Initially, Heuristic B, which demonstrated optimal performance in the original validation set, exhibits fragility within

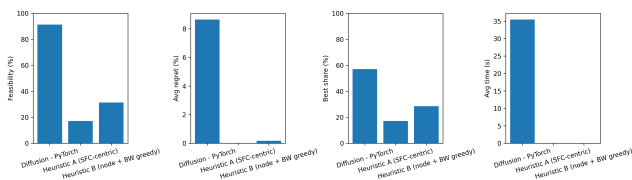


Fig. 10. Relative-cost evaluation on eval_change: feasibility, regret, best-share and runtime.

this constraint-tight regime. It successfully generates valid solutions in approximately one-third of the instances. In instances where this is a viable option, the cost remains predominantly optimal (average regret 0.18%), and frequently attains the most optimal cost (best-share 28.6%). However, in approximately 70% of cases, it is unsuccessful in identifying a valid placement. It is evident that Heuristic A is even more fragile, with a feasibility percentage of only 17.1%.

Secondly, the diffusion-based solver demonstrates significantly greater robustness. It maintains high feasibility (91.4%) in this challenging regime and attains optimal cost on 57.1% of the instances. The mean regret of approximately 8.7% of the optimal solution per instance is moderate, particularly given that the *optimal solution* is occasionally the heuristic and at other times diffusion itself.

Thirdly, the learning baselines demonstrate a complete collapse, with no capacity to generate a single viable solution on eval_change, despite having been trained on the original validation regime. Evidence suggests that subjects demonstrate substandard performance not only in relation to the base distribution, but also in terms of their ability to adapt to the tightened constraints.

To complement average regret values with a distributional view, Fig. 11 reports the empirical CDF of regret on eval_change for all feasible solutions returned by each method. This figure highlights not only which solver attains lower regret on average, but also how consistently each method stays close to the best solution across instances.

In summary, the evaluation on the eval_change dataset reveals a regime where the proposed diffusion-based solver outperforms the best heuristic when both feasibility and cost

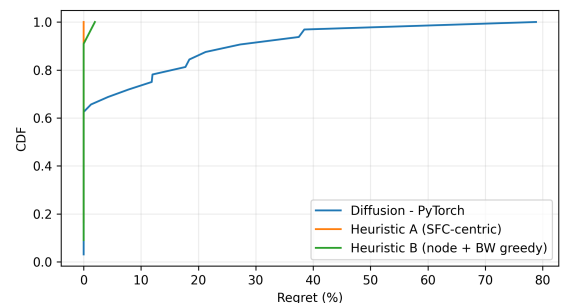


Fig. 11. CDF of regret on eval_change.

are considered. The heuristic remains near-optimal whenever it succeeds, but its success probability is low. In contrast, diffusion is feasible on almost all instances and often yields the best solution among all methods.

E. Discussion

The amalgamation of all experiments has resulted in the following key conclusions being supported by the extended evaluation. As demonstrated in [18], the original distribution, as well as larger instances (eval_big) and moderate cost shifts (eval_shift), employ a carefully engineered greedy heuristic (Heuristic B) that is of particular note. This heuristic attains near-optimal cost with very high feasibility and negligible runtime. In the aforementioned regimes, it has been demonstrated that the diffusion solver is significantly suboptimal in terms of cost. This finding calls into question the justification for its additional complexity and runtime.

It is evident that simple learning baselines (GDSG with GNN/MLP and supervised GNN) are not viable as solvers. These baselines produce feasible solutions on only a small percentage of instances in the base distribution, and they are unable to function effectively under the more challenging regime (eval_change).

However, when transitioning to a constraint-tight regime, characterised by stringent bandwidth and delay constraints (eval_change), the scenario undergoes a transformation. The heuristic, which previously exhibited optimal performance, becomes susceptible to disruption. Conversely, the proposed diffusion-based solver demonstrates a heightened degree of robustness, ensuring feasibility in approximately 91% of instances and attaining optimal cost in about 57% of them, accompanied by a moderate average regret.

The results obtained provide a compelling illustration of the underlying dynamics. In instances where a robust heuristic is in place and the constraints are relatively flexible, diffusion-based solvers encounter significant challenges in their ability to compete. However, when constraints are tightened and the global structure of the solution becomes more important than local, greedy decisions, a diffusion-based solver trained on optimal placements can provide robust, high-quality solutions where handcrafted rules break down. This is precisely the type of setting in which generative solvers, such as diffusion models, are considered to be advantageous for network optimisation. These solvers are not intended to replace simple heuristics in straightforward scenarios; rather, they are regarded as robust solutions in complex, constrained contexts where traditional heuristics are ineffective.

VI. CONCLUSION

In summary, we have adapted the DDPM framework to the CNF/SFC placement problem by:

- 1) encoding the network and CNFs as a graph input,
- 2) designing a GNN that predicts the Gaussian noise required for denoising steps,
- 3) integrating domain constraints as losses.

Each component is grounded in diffusion theory and network optimization literature. The use of a GraphSAGE encoder with edge features ensures node capacities and link data influence the embedding. The time embedding follows standard DDPM practice, allowing the network to calibrate its denoising at each step. The MLP decoder on concatenated pair embeddings is analogous to score networks in continuous DDPMs, but adapted here to output a score for each discrete choice.

This diffusion-based approach leverages the strengths of neural generative modelling for combinatorial optimization. The iterative denoising process allows the model to gradually refine a placement and effectively navigating the exponentially large solution space. By using a GNN, it captures the rich structure of the network and CNFs embedding global state in the latent vectors. Unlike greedy heuristics or standalone neural predictors, the diffusion model can produce diverse solutions by sampling, which is useful since the problem is NP-hard and multiple good solutions may exist.

In conclusion, the diffusion-based CNF placement solver demonstrates how generative modelling and graph neural networks can be combined to tackle network optimization. It bridges the theoretical foundations of DDPMs with the specific constraints of CNFs, yielding a practical end-to-end system.

A. Limitations

It is important to note that this study is also subject to limitations. Firstly, the quality of the solution is contingent upon best-of- k sampling, a process which engenders a controllable yet non-negligible inference cost as k increases. Secondly, the present training system is overseen by optimal (or near-optimal) solver-generated placements, which can be costly to acquire on larger scales. Incorporating training from suboptimal data or self-improvement loops is a logical subsequent step. Thirdly, the focus of this evaluation is on a static, cloud-only continuum and sequential-chain delay accounting. However, extending this to encompass richer continuum dynamics (e.g., time-varying links, multi-hop forwarding, and full DAG delay semantics) would serve to broaden its applicability.

B. Future work

Future research will examine the utilisation of more robust constraint satisfaction mechanisms during the sampling process, such as guidance, projection, or repair. Additionally, it will explore architectural variants that enhance cost optimality in scenarios where strong greedy heuristics already prevail.

REFERENCES

- [1] ETSI, "Network Functions Virtualisation (NFV); Management and Orchestration," European Telecommunications Standards Institute (ETSI), Tech. Rep. ETSI GS NFV-MAN 001 V1.1.1, Dec. 2014.
- [2] 3GPP, "System Architecture for the 5G System (5GS)," 3rd Generation Partnership Project (3GPP), Tech. Rep. TS 23.501, 2022.
- [3] ETSI, "Multi-access Edge Computing (MEC); Framework and Reference Architecture," European Telecommunications Standards Institute (ETSI), Tech. Rep. ETSI GS MEC 003 V4.1.1, May 2025.
- [4] F. Bonomi, R. Milito, P. Natarajan, and J. Zhu, "Fog computing and its role in the internet of things," in *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing (MCC)*, 2012.

- [5] D. Bhamare, R. Jain, M. Samaka, and A. Erbad, "A survey on service function chaining," *Journal of Network and Computer Applications*, vol. 75, pp. 138–155, Nov. 2016.
- [6] A. Laghrissi and T. Taleb, "A survey on the placement of virtual resources and virtual network functions," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1409–1434, 2019.
- [7] G. L. Santos, T. Lynn, J. Kelner, and P. T. Endo, "The greatest teacher, failure is: Using reinforcement learning for sfc placement based on availability and energy consumption," *arXiv preprint*, 2020, arXiv:2010.05711.
- [8] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., 2020.
- [9] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=PXTIG12RRHS>
- [10] Z. Sun and Y. Yang, "Difusco: graph-based diffusion solvers for combinatorial optimization," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [11] R. Liang, B. Yang, Z. Yu, B. Guo, X. Cao, M. Debbah, H. V. Poor, and C. Yuen, "Diffsg: A generative solver for network optimization with diffusion model," *IEEE Communications Magazine*, vol. 63, no. 6, pp. 16–24, Jun. 2025.
- [12] R. Liang, B. Yang, P. Chen, X. Li, Y. Xue, Z. Yu, X. Cao, Y. Zhang, M. Debbah, H. Vincent Poor, and C. Yuen, "Diffusion models as network optimizers: Explorations and analysis," *IEEE Internet of Things Journal*, vol. 12, no. 10, pp. 13 183–13 193, May 2025.
- [13] R. Liang, B. Yang, P. Chen, X. Cao, Z. Yu, M. Debbah, D. Niyato, H. V. Poor, and C. Yuen, "Gdsg: Graph diffusion-based solution generator for optimization problems in mec networks," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 10 264–10 277, Oct. 2025.
- [14] H. Du, R. Zhang, Y. Liu, J. Wang, Y. Lin, Z. Li, D. Niyato, J. Kang, Z. Xiong, S. Cui, B. Ai, H. Zhou, and D. I. Kim, "Enhancing deep reinforcement learning: A tutorial on generative diffusion models in network optimization," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 4, pp. 2611–2646, 2024.
- [15] Z. Zhang, V. Aggarwal, and T. Lan, "Network diffuser for placing-scheduling service function chains with inverse demonstration," in *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*. IEEE, May 2025, pp. 1–10.
- [16] A. Satpathy, M. N. Sahoo, C. Swain, P. Bellavista, M. Guizani, K. Muhammad, and S. Bakshi, "Virtual network embedding: Literature assessment, recent advancements, opportunities, and challenges," *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2025.
- [17] K. Doan, M. Avgeris, A. Leivadreas, I. Lambadaris, and W. Shin, "Cooperative learning-based framework for vnf caching and placement optimization over low earth orbit satellite networks," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 3, pp. 4758–4773, Mar. 2025.
- [18] A. Vázquez-Rodríguez, M. Fernández-Veiga, and C. Giraldo-Rodríguez, "Diffusion-based solver for cnf placement on the cloud-continuum," in *2025 International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, 2025, pp. 476–482.